Software Requirements Specification (SRS)

Project: Traffic Management System
Version: 1.0
Authors: Vijeth, Aarya, Vaibhav, Vainik
Date: 9-10-2026

Table of Contents

1. Introduction
2. Overall description
3. External interfaces
4. Functional requirements
5. Non-functional requirements
6. Quality Attributes & Acceptance Tests
7. UML Use-Case Diagram
8. Requirements Traceability Matrix (RTM)

Introduction

Purpose

This document is a Software Requirements Specification (SRS) for the Traffic Management System (Simulation). It defines the functional and non-functional requirements, external interfaces, system constraints, security requirements, and verification criteria for the proposed system.

The document serves as a reference for developers, testers, and evaluators throughout the design, implementation, integration, and testing phases of the project. It establishes a clear understanding of the system's expected behavior and provides a basis for verifying that the implemented system satisfies its specified requirements.

Scope

The Traffic Management System is a C++17-based, discrete-time simulation designed to model vehicle movement across a configurable road network containing roads, lanes, intersections, and traffic signals.

The system covers vehicle generation, lane-queue management, adaptive traffic-signal control, road-network representation, vehicle routing and movement, congestion detection, incident handling, emergency-vehicle priority, and traffic-performance reporting.

The system also supports deterministic simulation scenarios to facilitate repeatable testing and evaluation.

The project focuses on demonstrating modular software design, appropriate data structures and algorithms, and systematic software testing. It does not include real-world traffic-signal hardware control, live GPS or CCTV integration, machine-learning-based traffic prediction, or detailed vehicle physics.

Audience

This document is intended for the following stakeholders:

1. **Developers:** To understand the system requirements and implement the corresponding modules.
2. **Testers and QA Engineers:** To derive test cases, verify expected behavior, and validate compliance with the requirements.
3. **System Integrators:** To understand interactions and dependencies between vehicle management, signal control, road-network management, and incident-management modules.
4. **Project Maintainers:** To understand the system's intended functionality, constraints, and modular organization.
5. **Assessment Evaluators:** To assess the completeness of the requirements, implementation, testing, and validation activities.

Definitions and Acronyms

Term	Definition
TMS	Traffic Management System.
SRS	Software Requirements Specification.
FR	Functional Requirement.
NFR	Non-Functional Requirement.
RTM	Requirements Traceability Matrix.
UML	Unified Modeling Language.
C++	Programming language used to implement the simulation.
CMake	Build-system generator used to configure and build the project.
Simulation Tick	A discrete unit of simulated time during which system events and traffic states are updated.
Lane Queue	An ordered collection of vehicles waiting to move through a lane.
Intersection	A junction where two or more roads connect.
Signal Phase	A defined traffic-signal state that permits specified vehicle movements.
Adaptive Signal Control	Adjustment of signal timing based on traffic conditions, such as queue lengths.
Emergency Priority	A controlled mechanism for prioritizing emergency-vehicle movement while preserving signal safety.
Road Network	A graph representing intersections and connecting roads.
Congestion	A traffic condition identified using configured queue-length or road-occupancy thresholds.

Incident	An event, such as a road blockage, that makes a road unavailable or restricts its use.
Throughput	The number of vehicles completing a specified movement or journey during a given interval.
Deterministic Seed	A seed used to reproduce pseudo-random vehicle-generation behavior across simulation runs.
CSV	Comma-Separated Values, a format used to export simulation results.

Overall Description

Product Perspective

The Traffic Management System is a standalone, discrete-event or time-stepped traffic simulation developed using C++17 and CMake. It models road networks, intersections, traffic signal phases, vehicle movement, congestion, and road incidents.

The system allows users to create or load a simulation scenario, generate vehicles according to configurable traffic arrival rates or deterministic schedules, and simulate their movement through a connected road network. Signal timing and traffic management policies control vehicle movement while respecting lane capacities and intersection safety constraints.

The simulation collects traffic performance metrics, including throughput, waiting time, queue lengths, and completed or unfinished trips. Results are displayed through readable console output and can be exported as a CSV summary.

The system is intended for academic demonstration, testing traffic management strategies, and comparing the effects of different traffic conditions and control policies. It does not directly control physical traffic signals or real-world vehicles.

Major Product Functions

1. Create or load traffic simulation scenarios.
2. Configure intersections, roads, lane capacities, and simulation duration.
3. Generate vehicles using configurable arrival rates or deterministic schedules.
4. Assign unique identifiers, vehicle classes, source and destination nodes, and routes.
5. Maintain vehicle queues while enforcing lane capacity constraints.
6. Represent the road network using connected nodes and edges.
7. Manage traffic signal phases and safely advance signal timers.

8. Adapt green signal duration using queue information within configured limits.
9. Support controlled emergency-priority requests.
10. Move vehicles through intersections and along their planned routes.
11. Prevent vehicles from entering full downstream lanes or roads.
12. Select routes using shortest-path or weighted-path algorithms.
13. Detect congestion using queue length or occupancy thresholds.
14. Simulate road blockages and traffic incidents.
15. Trigger rerouting or traffic-management responses when required.
16. Collect throughput, waiting time, queue statistics, and trip completion metrics.
17. Export a final simulation report in text or CSV format.
18. Support deterministic seeds and configurations for reproducible simulations.

User Roles and Characteristics

- *Scenario Designer*: Creates or loads scenarios and configures intersections, roads, capacities, vehicle generation, and simulation duration.
- *Simulation Operator*: Runs, monitors, and terminates simulations and observes signal phases, vehicle movement, and congestion.
- *Traffic Management Analyst*: Examines performance metrics and evaluates the effects of signal timing, rerouting, and incident handling.
- *Developer or Tester*: Tests individual modules, validates safety constraints, investigates failures, and verifies reproducibility.

Operating Environment

- C++17-compatible compiler.
- CMake build system.
- Linux operating system.
- Standard command-line interface or console menu.
- Local configuration files and simulation input data.
- Local file system for text reports, CSV exports, and test results.
- Standard student laptop capable of running the specified demonstration scenarios.

Constraints

- The implementation must use C++17 and CMake.
- The simulation must enforce configured road and lane capacities.
- Conflicting green signal phases must never be enabled simultaneously at the same controlled intersection.
- Emergency-priority requests must preserve intersection safety constraints.
- Invalid configurations and impossible capacity or timing values must be rejected.
- Random vehicle generation must support deterministic seeds.
- Modules must communicate through clearly defined interfaces.
- The system must not depend on physical traffic controllers or external traffic infrastructure.

External Interface Requirements

User Interfaces

The system shall provide a console-based menu or command interface through which users can:

- Create or load a scenario.
- Configure simulation parameters.
- Start and terminate a simulation.
- Inspect traffic and signal status.
- Inject supported road incidents.
- Submit emergency-priority requests.
- View simulation statistics.
- Export final reports.

Console messages, errors, metric labels, and menu options shall be readable and consistent.

Hardware Interfaces

The system shall operate on a general-purpose computer and shall not require specialized traffic-control hardware.

Software Interfaces

- *Scenario Configuration Interface*: Reads scenario definitions, including intersections, roads, capacities, simulation duration, and vehicle generation parameters.
- *Road Network Interface*: Provides access to connected nodes, road edges, capacities, and travel times.
- *Vehicle Management Interface*: Creates vehicles, assigns unique IDs and routes, and maintains vehicle states.
- *Signal Control Interface*: Maintains signal phases, advances timers, and evaluates adaptive timing and emergency-priority requests.
- *Traffic Management Interface*: Evaluates congestion and incidents and initiates supported rerouting or management responses.
- *Metrics and Reporting Interface*: Collects simulation statistics and produces text or CSV reports.
- *Build and Test Interface*: Uses CMake to configure the project and supports module-level and integration testing.

Communications

The system shall communicate through in-process C++ module interfaces. Modules shall exchange validated data through defined APIs or data structures, avoiding uncontrolled global mutable state outside a controlled simulation context.

System features

Req ID	Requirement	Priority	Source/ Stakeholder	Acceptance Criteria / Test Case Ref	Comments / Dependencies
FR-01	Create or load a simulation scenario containing intersections, roads, lane capacities, and run duration.	High	Aarya / Integration	Valid scenarios load successfully; invalid configurations are rejected with clear errors. TC-01	Required by all simulation modules.
FR-02	Generate vehicles using configurable arrival rates or a deterministic test schedule.	High	Aarya	Vehicles are generated according to configured arrival rates or schedules. TC-02	Depends on scenario configuration and random seed settings.
FR-03	Assign each generated vehicle a unique ID, source, destination, class, and route information.	High	Aarya	Every vehicle has a unique ID and valid required attributes. TC-03	Depends on vehicle generation and road network.
FR-04	Place vehicles into valid entry-lane queues without exceeding lane capacity.	High	Aarya	No entry lane exceeds its configured capacity. TC-04	Depends on lane definitions and vehicle generation.
FR-05	Maintain signal phases for every controlled intersection.	High	Vijeth	Every controlled intersection maintains a valid signal state. TC-05	Depends on intersection configuration.
FR-06	Advance signal timers and switch phases safely.	High	Vijeth	Phase transitions follow timing rules and never enable conflicting green phases. TC-06	Depends on FR-05 and signal safety rules.
FR-07	Use queue information to extend or shorten a green phase within configured limits.	Medium	Vijeth	Green duration responds to queue conditions while respecting minimum and maximum limits. TC-07	Depends on FR-05, FR-06, and queue statistics.
FR-08	Provide a controlled emergency-priority request that never enables conflicting greens.	Medium	Vijeth	Valid requests trigger priority handling while preserving intersection safety. TC-08	Depends on signal control and safety validation.
FR-09	Represent the road network as connected nodes and edges with capacities and travel times.	High	Vaibhav	Road connections, capacities, and travel times are valid and accessible. TC-09	Required by routing and vehicle movement.

FR-10	Move eligible vehicles through green intersections and along their planned routes.	High	Vaibhav	Vehicles move only when signal state, route connectivity, and downstream capacity permit. TC-10	Depends on FR-05, FR-06, and FR-09.
FR-11	Prevent movement into a full downstream road or lane.	High	Vaibhav	Downstream road and lane occupancy never exceeds configured capacity. TC-11	Depends on road network and vehicle movement.
FR-12	Select a route using a simple shortest-path or weighted-path policy.	Medium	Vaibhav	The selected route uses connected edges and minimizes the configured route cost. TC-12	Depends on FR-09 and destination information.
FR-13	Measure lane or intersection congestion from queue length or occupancy thresholds.	High	Vainik	Congestion is detected when configured thresholds are met. TC-13	Depends on queue statistics and capacity information.
FR-14	Inject a road blockage or incident and mark affected road segments unavailable or restricted.	Medium	Vainik	Affected segments reflect the incident state and prohibited movements are prevented. TC-14	Depends on road network representation.
FR-15	Trigger a rerouting or traffic-management response when an incident or congestion condition is detected.	Medium	Vainik	The configured response activates when trigger conditions are met; no-route situations are handled safely. TC-15	Depends on FR-12, FR-13, and FR-14.
FR-16	Collect simulation metrics including throughput, waiting time, queue statistics, and completed or unfinished trips.	High	Vainik	Metrics correctly reflect recorded simulation events and vehicle states. TC-16	Depends on vehicle movement and simulation event tracking.
FR-17	Print or export a final text or CSV summary for the completed scenario.	Medium	Vainik	The final report contains required metrics in a readable text or CSV format. TC-17	Depends on FR-16 and file output handling.
FR-18	Support deterministic seeds and configurations so tests and demonstrations are reproducible.	High	Shared	Identical configurations and seeds produce equivalent results under deterministic execution conditions. TC-18	Shared requirement across all modules.

Non-functional requirements

Req ID	Requirement	Type	Priority	Source/Stakeholder	Acceptance Criteria / Test Case Ref
NFR-01	Core modules must communicate through clear interfaces; avoid global mutable state except a controlled simulation context.	Non-functional — Maintainability	High	Development team / Integration	Module interfaces are clearly defined and uncontrolled global mutable state is absent. TC-NFR-01
NFR-02	Each module must expose deterministic methods that can be tested without running the entire simulation.	Non-functional — Testability	High	Development team / Testers	Unit tests execute individual modules independently and verify expected outputs. TC-NFR-02
NFR-03	A standard demo scenario of at least 4 intersections and 500 generated vehicles should complete comfortably on a normal student laptop without noticeable hangs.	Non-functional — Performance	High	Development team / Evaluator	The benchmark scenario completes without noticeable hangs; execution time is recorded. TC-NFR-03
NFR-04	Invalid configuration must be rejected with clear error messages rather than causing undefined behavior.	Non-functional — Reliability	High	Development team / Scenario designer	Invalid capacities, timings, and network references are rejected without crashes or undefined behavior. TC-NFR-04
NFR-05	Build using C++17 and CMake on Linux; avoid	Non-functional — Portability	High	Development team / Build	The project configures and builds successfully in a supported Linux environment

	compiler-specific extensions.			engineer	using C++17 and CMake. TC-NFR-05
NFR-06	Console commands, menus, and report labels should be readable and consistent.	Non-functional — Usability	Medium	Simulation operator / Evaluator	Users can identify commands, understand error messages, and interpret simulation reports. TC-NFR-06
NFR-07	Validate all file/menu inputs, reject impossible capacities/times, avoid unsafe C-style string handling, and use RAII.	Non-functional — Security / Robustness	High	Development team / Testers	Invalid inputs are rejected, unsafe string-handling practices are avoided, and resources are managed using RAII. TC-NFR-07
NFR-08	Every implemented FR must map to source files and test IDs in the traceability matrix.	Non-functional — Traceability	High	Development team / Project evaluator	Every implemented FR has corresponding source-file references and test IDs. TC-NFR-08
NFR-09	Warnings enabled, formatting consistent, static analysis clean for critical findings.	Non-functional — Code quality	High	Development team / Code reviewer	Compiler warnings are enabled, formatting is consistent, and no critical static-analysis findings remain unresolved. TC-NFR-09
NFR-10	Random traffic generation must accept a seed and print/store the seed used.	Non-functional — Reproducibility	High	Development team / Tester	The seed is recorded, and repeated runs with the same seed and configuration produce equivalent traffic and results. TC-NFR-10

Quality Attributes & Acceptance Tests

Quality Attributes

- **Correctness:** Vehicles must follow valid routes, obey traffic signal phases, and respect road and lane capacities.
- **Safety:** Conflicting green signal phases must never occur, including during emergency-priority requests.
- **Performance:** A scenario with at least four intersections and 500 generated vehicles must complete on a normal student laptop without noticeable hangs.
- **Reliability:** Invalid configurations, road blockages, and unavailable routes must be handled without crashes or undefined behavior.
- **Maintainability and Testability:** Modules must have clear interfaces and support independent unit testing.
- **Portability:** The system must build using C++17 and CMake on Linux.
- **Reproducibility:** Identical scenarios and random seeds must produce equivalent results under deterministic execution conditions.
- **Usability:** Console menus, error messages, simulation status, and exported reports must be clear and consistent.

Acceptance Tests

The following acceptance test suites shall be executed:

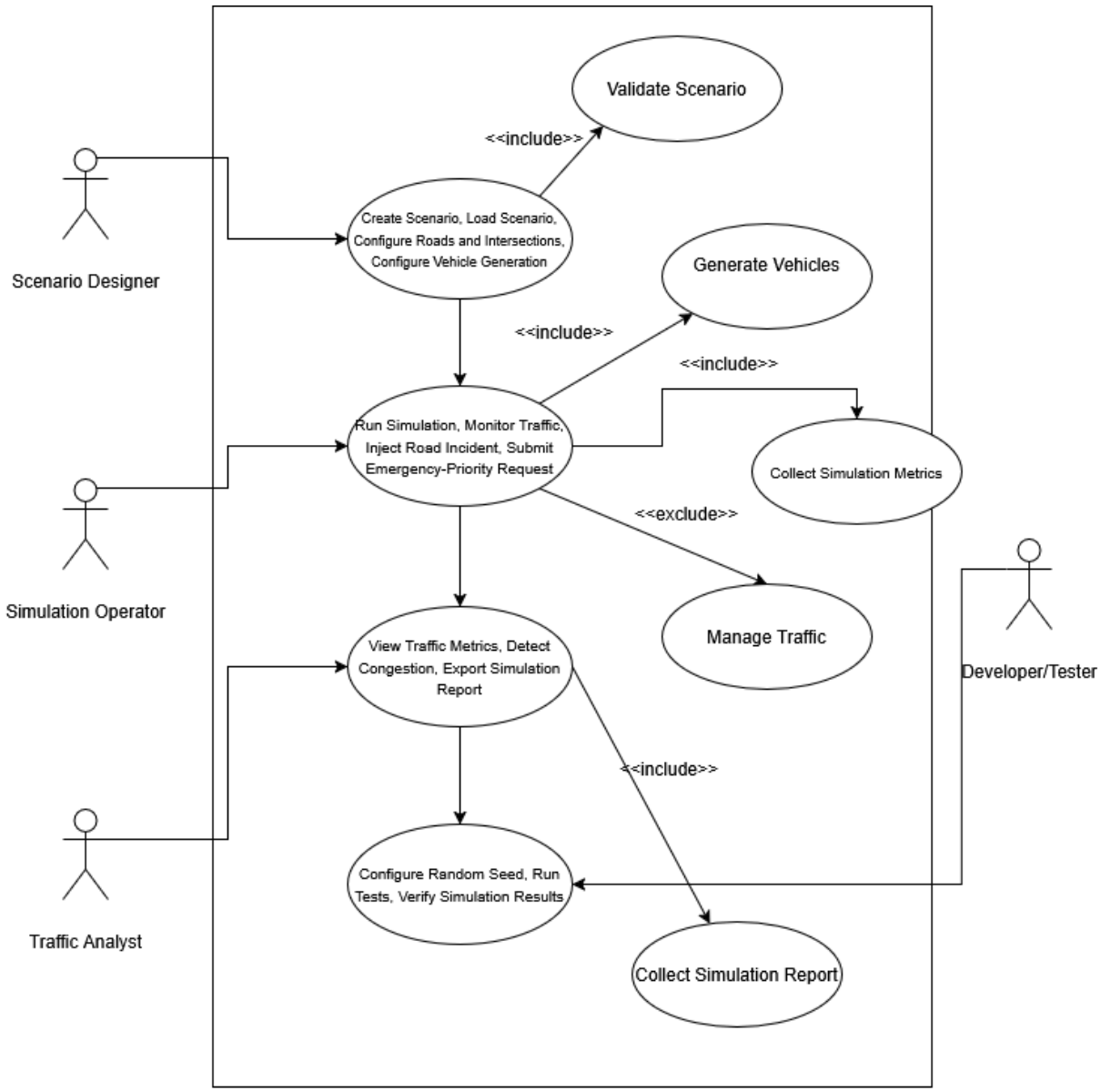
1. **Scenario Configuration Tests:** Verify scenario creation/loading, configuration validation, and rejection of invalid inputs.
2. **Vehicle Generation and Movement Tests:** Verify vehicle attributes, unique IDs, queue management, valid movement, and capacity enforcement.
3. **Traffic Signal Control Tests:** Verify signal phase transitions, timer limits, adaptive green durations, and prevention of conflicting green phases.
4. **Routing and Incident Management Tests:** Verify route selection, congestion detection, road blockage handling, and rerouting behavior.
5. **Emergency-Priority Tests:** Verify that emergency-priority requests are handled correctly without violating signal safety constraints.
6. **Performance and Metrics Tests:** Execute the standard scenario with at least four intersections and 500 vehicles; verify execution behavior and correctness of throughput, waiting time, queue statistics, and trip counts.
7. **Reporting and Reproducibility Tests:** Verify text/CSV report generation and equivalent results across repeated runs with the same configuration and seed.
8. **Reliability, Portability, and Code Quality Tests:** Verify error handling, successful C++17/CMake builds on Linux, resource management, and the absence of unresolved critical static-analysis findings.

Exit Criteria

The system shall be accepted when:

- All high-priority functional requirements have been implemented and verified.
- All critical safety constraints, particularly signal conflict prevention and road-capacity enforcement, pass their tests.
- No critical non-functional requirement failures remain unresolved.
- All applicable test cases in the Requirements Traceability Matrix (RTM) have passed.
- The standard performance scenario completes successfully.
- Simulation reports are generated correctly, and reproducibility tests pass.

UML Use-Case diagram



Requirements Traceability Matrix (RTM)

Req ID	Requirement Short	Module	Test Case(s)	Status (N/P/A)	Comments
FR-01	Create/load scenario	Scenario Module	TC-01	N	Depends on configuration validation
FR-02	Generate vehicles	Vehicle Generation Module	TC-02	N	Supports arrival rates and deterministic schedules
FR-03	Assign vehicle attributes	Vehicle Module	TC-03	N	Unique IDs and valid attributes
FR-04	Manage entry queues	Queue Management Module	TC-04	N	Enforces lane capacity
FR-05	Maintain signal phases	Signal Control Module	TC-05	N	Valid state for each intersection
FR-06	Advance signal timers safely	Signal Control Module	TC-06	N	Prevents conflicting green phases
FR-07	Adapt green duration	Adaptive Signal Module	TC-07	N	Respects configured timing limits
FR-08	Handle emergency priority	Emergency Priority Module	TC-08	N	Maintains signal safety constraints
FR-09	Represent road network	Road Network Module	TC-09	N	Nodes, edges, capacities, and travel times
FR-10	Move vehicles through network	Vehicle Movement Module	TC-10	N	Checks signals, routes, and capacity
FR-11	Enforce downstream capacity	Vehicle Movement Module	TC-11	N	Prevents capacity overflow
FR-12	Select vehicle routes	Routing Module	TC-12	N	Shortest-path or weighted-path policy
FR-13	Detect congestion	Congestion Detection	TC-13	N	Uses queue or occupancy thresholds

		Module			
FR-14	Simulate road incidents	Incident Management Module	TC-14	N	Supports road blockage and restrictions
FR-15	Respond to congestion/incidents	Traffic Management Module	TC-15	N	Rerouting and no-route handling
FR-16	Collect simulation metrics	Metrics Module	TC-16	N	Throughput, waiting time, queues, and trips
FR-17	Export simulation report	Reporting Module	TC-17	N	Text and CSV output
FR-18	Ensure reproducibility	Simulation Controller	TC-18	N	Seed and configuration controlled
NFR-01	Maintainability	All Modules	TC-NFR-01	N	Clear interfaces; controlled shared state
NFR-02	Testability	All Modules	TC-NFR-02	N	Independent deterministic unit tests
NFR-03	Performance	Simulation Engine	TC-NFR-03	N	At least 4 intersections and 500 vehicles
NFR-04	Reliability	Scenario Module	TC-NFR-04	N	Rejects invalid configurations safely
NFR-05	Portability	Build System	TC-NFR-05	N	C++17, CMake, and Linux
NFR-06	Usability	Console Interface	TC-NFR-06	N	Readable menus and reports
NFR-07	Security and robustness	Input Validation / Core	TC-NFR-07	N	Input validation and RAII
NFR-08	Traceability	Project Documentation	TC-NFR-08	N	Maps FRs to code and tests
NFR-09	Code quality	Build and Analysis Tools	TC-NFR-09	N	Warnings and static analysis
NFR-10	Reproducibility	Vehicle Generation / Simulation Controller	TC-NFR-10	N	Records and reuses random seed